Séance 4 — Préparer l'application au monde réel

Sommaire

1. Concurrence
   - 1.1 🟢 Pourquoi la concurrence ?
   - 1.2 🟢 Lancer des tâches : `ExecutorService`
   - 1.3 🟢 Les threads virtuels
   - 1.4 🟢 Race condition : le compteur qui perd des commandes
   - 1.5 🟢 Correction 1 : `AtomicInteger`
   - 1.6 🟢 Correction 2 : `synchronized`
   - 1.7 🟢 `ConcurrentHashMap` et ses opérations atomiques
   - 1.8 🔒 TOCTOU : « vérifier puis agir »
   - 1.9 🔒 TOCTOU sur le stock : la dernière portion
   - 1.10 🔵 Deadlocks
   - 1.11 🔵 🔒 TOCTOU sur les fichiers
   - 1.12 🟣 Scoped values (JEP 506, final en Java 25)
   - 1.13 🟣 À venir : la concurrence structurée (preview)
2. Persistance avec JDBC
   - 2.1 🟢 Pourquoi une base de données ? JDBC et H2
   - 2.2 🟢 Se connecter : `Connection` et try-with-resources
   - 2.3 🔒 Identifiants hors du code et moindre privilège
   - 2.4 🟢 Créer le schéma : du SQL lisible avec les text blocks
   - 2.5 🟢 `PreparedStatement` : écrire une commande
   - 2.6 🟢 `ResultSet` : relire et mapper vers des objets
   - 2.7 🔒 Injection SQL
   - 2.8 🔒 Les limites de `PreparedStatement` : les noms de colonnes
   - 2.9 🔵 Transactions
3. 🔒 Sécurité : pour aller plus loin
   - 3.1 🔵 🔒 Stocker des mots de passe : PBKDF2
   - 3.2 🟣 🔒 L'API KDF (JEP 510, finale en Java 25) : dériver des clés, pas hacher des mots de passe
   - 3.3 🔒 Désérialisation Java : ne désérialisez jamais une donnée non fiable
4. Ce qu'il faut retenir
5. Pour aller plus loin

> **Objectifs** : AA7 — identifier et corriger une race condition et une injection SQL. Plus précisément, à la fin de cette séance, vous savez :
>
> - lancer des tâches concurrentes avec un `ExecutorService` et des threads virtuels ;
> - reconnaître une race condition et la corriger (`AtomicInteger`, `synchronized`, `ConcurrentHashMap`) ;
> - reconnaître un TOCTOU (« vérifier puis agir ») et le rendre atomique ;
> - lire et écrire en base avec JDBC (`Connection`, `PreparedStatement`, `ResultSet`) ;
> - démontrer une injection SQL et la corriger ; garder les secrets hors du code.
>
> **Prérequis** : S2 (modèle FoodFast, `Map`, `record`, `equals` / `hashCode`), S3 (lambdas, exceptions, try-with-resources, `System.Logger`).
>
> **Comment lire ce support** : 🟢 indispensable · 🔵 pour approfondir · 🟣 bonus · 🔒 sécurité

Jusqu'ici, FoodFast vivait dans un monde idéal : un seul utilisateur, et des données qui disparaissent à l'arrêt du programme. Cette séance traite les deux problèmes du « monde réel » :

1. **Plusieurs clients en même temps** : c'est la concurrence. Le bug typique est la *race condition* ; la faille typique est le *TOCTOU*.
2. **Des données qui survivent au redémarrage** : c'est la persistance avec JDBC. La faille typique est l'*injection SQL*.

Tous les exemples marqués `// Fichier :` sont compilés et exécutés avec Java 25 et H2 2.x. Les exemples JDBC utilisent **H2 en mémoire** : aucune installation, la base vit le temps du programme.

---

1. Concurrence

1.1 🟢 Pourquoi la concurrence ?

Le problème. Vendredi, 19 h 55. Des centaines de clients valident leur panier dans la même seconde. Pour chaque commande, la plateforme doit :

- prévenir le restaurant (appel réseau, environ 100 ms) ;
- enregistrer la commande en base (environ 10 ms) ;
- envoyer un e-mail de confirmation (environ 200 ms).

Traitées **une par une**, 1 000 commandes demandent plus de 5 minutes. Le 1 000e client attend 5 minutes devant son écran. Pendant ces 300 ms par commande, le processeur ne fait **rien** : il attend le réseau.

La solution : traiter plusieurs commandes **en même temps**, chacune dans son propre *thread* (fil d'exécution). Un serveur web Java fait exactement cela : une requête HTTP, un thread.

Le prix à payer : les threads d'un même programme **partagent la mémoire** (le tas). Deux threads qui modifient le même objet au même moment peuvent le corrompre. C'est le sujet de toute cette partie.

> Concurrence et parallélisme ne sont pas synonymes. La **concurrence** consiste à gérer plusieurs tâches en cours en même temps, éventuellement en les entrelaçant sur un seul cœur. Le **parallélisme** consiste à exécuter

> réellement plusieurs calculs au même instant, sur plusieurs cœurs. Les bugs de cette séance apparaissent dès qu'il y a concurrence.

1.2 🟢 Lancer des tâches : `ExecutorService`

Pourquoi pas `new Thread(...)` ? On peut créer un thread à la main :

```
Thread t = new Thread(() -> notifyRestaurant(order));
t.start();
```

Mais il faut ensuite attendre chaque thread (`join()`), gérer les erreurs de chacun, ne pas en oublier… Dans un vrai programme, on confie ce travail à un `ExecutorService` : on lui soumet des tâches, il les exécute et gère leur cycle de vie.

Comment ? Depuis Java 19, `ExecutorService` est `AutoCloseable` : on l'utilise dans un **try-with-resources**, comme une connexion ou un fichier (S3). À la sortie du bloc, `close()` **attend la fin de toutes les tâches soumises**, puis libère les threads. Plus besoin de `shutdown()` + `awaitTermination()` + gestion d'interruption.

```
// Fichier : NotifyRestaurants.java
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

// Simule l'appel réseau au restaurant : 100 ms d'attente, puis un accusé de réception.
String notifyRestaurant(int orderNumber) throws InterruptedException {
    Thread.sleep(Duration.ofMillis(100));
    return "order-" + orderNumber + " reçue par le restaurant";
}

void main() throws Exception {
    long start = System.nanoTime();
    List<Future<String>> receipts = new ArrayList<>();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int i = 1; i <= 10_000; i++) {
            int n = i; // une lambda ne capture que des variables (effectivement) finales
            receipts.add(executor.submit(() -> notifyRestaurant(n)));
        }
    } // close() attend que les 10 000 tâches soient terminées

    long ms = Duration.ofNanos(System.nanoTime() - start).toMillis();
    IO.println(receipts.size() + " restaurants prévenus en " + ms + " ms");
    IO.println("Dernier accusé : " + receipts.getLast().get());
}
```

Sortie typique (la durée varie selon la machine) :

```
10000 restaurants prévenus en 127 ms
Dernier accusé : order-10000 reçue par le restaurant
```

10 000 appels de 100 ms chacun, soit 1 000 secondes en séquentiel, terminés en moins d'une seconde.

Quelques points à noter :

- `submit(...)` renvoie un `Future<T>` : une promesse de résultat. `future.get()` attend le résultat, ou relance l'exception de la tâche enveloppée dans une `ExecutionException`.
- Une lambda qui **renvoie une valeur** est un `Callable<T>` : elle a le droit de lever des exceptions vérifiées (ici `InterruptedException`). Une lambda qui ne renvoie rien est un `Runnable` : pas d'exception vérifiée.

> ❌ **Piège : l'exception avalée.** Si une tâche soumise avec `submit` lève une exception et que personne n'appelle `get()` sur son `Future`, l'exception est **perdue en silence**. C'est exactement ce que S3 interdit (« ne jamais avaler une exception »). ✅ Récupérez les `Future` et appelez `get()`, ou bien gérez et journalisez l'erreur **dans** la tâche.

1.3 🟢 Les threads virtuels

Pourquoi ? Un thread « classique » (dit **thread plateforme**) correspond à un thread du système d'exploitation. Il coûte environ 1 Mo de pile, et sa création est lente. On ne peut pas en créer des centaines de milliers. D'où l'habitude historique des *pools* de taille fixe (`Executors.newFixedThreadPool(10)`) : 10 threads se partagent toutes les tâches. Avec 10 threads et des tâches de 100 ms, l'exemple précédent prendrait 100 secondes.

Comment ? Un **thread virtuel** (Java 21, JEP 444, final) est un thread géré par la JVM, pas par le système. Il coûte quelques centaines d'octets. Quand il **attend** (réseau, base de données, `sleep`), la JVM le « décroche » du thread système qui le portait, et ce thread système exécute un autre thread virtuel pendant ce temps. On peut donc en créer **des millions**.

`Executors.newVirtualThreadPerTaskExecutor()` crée **un nouveau thread virtuel par tâche**.

Tâche	Bon choix	Pourquoi
Bloquante : réseau, JDBC, fichiers, appels HTTP	**Threads virtuels**	Le thread passe son temps à attendre : autant qu'il ne coûte rien
Calcul intensif : chiffrement, compression, calcul	Pool fixe, `newFixedThreadPool(nombre de cœurs)`	Plus de threads que de cœurs n'accélère pas le calcul

Règles pratiques :

- **On ne met pas les threads virtuels en pool** : ils sont jetables, on en crée un par tâche.
- Ce qu'on limite, c'est la **ressource rare** derrière : le nombre de connexions à la base (taille du pool de connexions), le nombre d'appels simultanés à une API (un `Semaphore`).
- Depuis le JDK 24 (JEP 491), un thread virtuel qui attend dans un bloc `synchronized` libère bien son thread porteur. L'ancien conseil « remplacer `synchronized` par `ReentrantLock` avec les threads virtuels » est donc **obsolète** en Java 25.

> Les threads virtuels rendent la concurrence **bon marché**. Ils ne la rendent pas **sûre** : une race condition reste une race condition, virtuelle ou non.

1.4 🟢 Race condition : le compteur qui perd des commandes

Le problème. La plateforme compte les commandes passées, pour le tableau de bord. Mille clients passent chacun mille commandes (c'est un test de charge). Le compteur devrait afficher 1 000 000.

```java
// Fichier : RaceDemo.java
import java.util.concurrent.Executors;

class OrderCounter {
    private int count = 0;

    void increment() {
        count++; // ❌ lire, ajouter 1, écrire : trois étapes, pas une
    }

    int get() {
        return count;
    }
}

void main() {
    var counter = new OrderCounter();
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int t = 0; t < 1_000; t++) {
            executor.submit(() -> {
                for (int i = 0; i < 1_000; i++) counter.increment();
            });
        }
    } // ici, toutes les tâches sont terminées : pas de sleep « au hasard »
    IO.println("Commandes comptées : " + counter.get() + " (attendu : 1000000)");
}
```

Trois exécutions successives sur une machine à 12 cœurs :

```
Commandes comptées : 174491 (attendu : 1000000)
Commandes comptées : 247164 (attendu : 1000000)
Commandes comptées : 152176 (attendu : 1000000)
```

La moitié des commandes, ou davantage, ont disparu, et le résultat change à chaque exécution. Avec seulement 2 cœurs (`java -XX:ActiveProcessorCount=2 RaceDemo.java`), on en perd encore environ 10 %.

Pourquoi ? `count++` ressemble à une instruction unique, mais la machine exécute trois étapes : **lire** `count` , **calculer** `count + 1` , **écrire** le résultat. Si deux threads s'entrelacent entre ces étapes, un incrément est écrasé :

```
count = 41
Thread A                    Thread B
lit count (41)
                            lit count (41)
calcule 41 + 1 = 42
                            calcule 41 + 1 = 42
écrit 42
                            écrit 42      ← l'incrément de A est perdu
count = 42 (au lieu de 43)
```

C'est une **race condition** (condition de course) : le résultat dépend de l'ordre, imprévisible, dans lequel les threads s'exécutent.

> ❌ **Piège : « chez moi, ça marche ».** Une race condition ne se manifeste pas forcément à chaque exécution. Avec 10 threads et 10 incréments, vous obtiendrez souvent le bon résultat. Avec un seul cœur, les threads virtuels ne s'entrelacent même pas au milieu de la boucle (essayez `java -XX:ActiveProcessorCount=1`

> `RaceDemo.java` : le compte est juste). Le bug est pourtant là, et il sortira en production, un vendredi soir. **Un test qui passe ne prouve pas l'absence de race condition.**

> ❌ **Piège : `Thread.sleep(2000)` pour « attendre la fin ».** L'ancienne version de ce cours démarrait 1 000 threads, dormait 2 secondes, puis lisait le compteur. Ce test affichait presque toujours 1 000 : chaque thread finissait son unique incrément avant que le suivant ne démarre. La démonstration ne démontrait rien. Il faut **beaucoup d'opérations par thread** pour que les threads se chevauchent, et un **vrai point d'attente** (`close()` de l'executor, ou `join()`), jamais un `sleep` .

Il y a deux familles de corrections : rendre l'opération **atomique** (§1.5), ou **interdire** à deux threads d'être dans la section critique en même temps (§1.6).

1.5 🟢 Correction 1 : `AtomicInteger`

Comment ? Le paquet `java.util.concurrent.atomic` fournit des types dont les opérations sont **atomiques** : elles s'exécutent en une seule étape indivisible, du point de vue des autres threads. Le processeur offre pour cela une instruction dédiée (*compare-and-swap*).

```
// Fichier : AtomicCounterDemo.java
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

class OrderCounter {
    private final AtomicInteger count = new AtomicInteger();

    void increment() {
        count.incrementAndGet(); // ✅ lire + ajouter + écrire en une seule opération atomique
    }

    int get() {
        return count.get();
    }
}

void main() {
    var counter = new OrderCounter();
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int t = 0; t < 1_000; t++) {
            executor.submit(() -> {
                for (int i = 0; i < 1_000; i++) counter.increment();
            });
        }
    }
    IO.println("Commandes comptées : " + counter.get() + " (attendu : 1000000)");
}
```

```
Commandes comptées : 1000000 (attendu : 1000000)
```

Méthodes utiles d'`AtomicInteger` (et d'`AtomicLong`) :

Méthode	Effet
`incrementAndGet()` / `decrementAndGet()`	`++x` / `--x` atomiques

Méthode	Effet
`addAndGet(delta)`	`x += delta` atomique
`updateAndGet(x -> …)`	applique une fonction, de manière atomique
`compareAndSet(attendu, nouveau)`	écrit `nouveau` seulement si la valeur vaut encore `attendu`

Quand l'utiliser ? Pour **une seule variable** : un compteur, un identifiant séquentiel. Dès que l'invariant porte sur **plusieurs variables** (« le total et le nombre de commandes doivent rester cohérents »), deux `AtomicInteger` ne suffisent plus : il faut un verrou (§1.6).

> 🔵 Pour un compteur **très** sollicité (des millions d'incréments par seconde, beaucoup de cœurs), `LongAdder` est plus rapide qu' `AtomicLong` : il répartit les incréments sur plusieurs cellules et ne fait la somme qu'à la lecture.

1.6 🟢 Correction 2 : `synchronized`

Comment ? Le mot-clé `synchronized` pose un **verrou** sur une portion de code. Chaque objet Java possède un verrou (son *moniteur*). Une méthode `synchronized` prend le verrou de `this` en entrée et le rend en sortie, même en cas d'exception. Tant qu'un thread détient le verrou, les autres attendent.

Le verrou garantit aussi la **visibilité** : ce qu'un thread a écrit avant de rendre le verrou est vu par le thread qui le prend ensuite.

Exemple FoodFast : des statistiques où deux valeurs doivent rester **cohérentes entre elles** (le nombre de commandes et le chiffre d'affaires). `AtomicInteger` ne suffit pas ici.

```java
// Fichier : SalesStatsDemo.java
import java.math.BigDecimal;
import java.util.concurrent.Executors;

class SalesStats {
    private int orderCount = 0;
    private BigDecimal revenue = BigDecimal.ZERO;

    // ✅ Un seul thread à la fois exécute une méthode synchronized de CET objet.
    synchronized void record(BigDecimal orderTotal) {
        orderCount++;
        revenue = revenue.add(orderTotal);
    }

    // ✅ La lecture prend le même verrou : on ne lit jamais un état « à moitié mis à jour ».
    synchronized String summary() {
        return orderCount + " commandes, " + revenue + " €";
    }
}

void main() {
    var stats = new SalesStats();
    var total = new BigDecimal("12.50");
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int t = 0; t < 1_000; t++) {
            executor.submit(() -> {
                for (int i = 0; i < 100; i++) stats.record(total);
            });
        }
    }
    IO.println(stats.summary()); // 100000 commandes, 1250000.00 €
}
```

On peut aussi verrouiller un **bloc** plutôt qu'une méthode entière, sur un objet verrou privé. C'est la forme recommandée dans une classe publique : un code extérieur ne peut pas prendre ce verrou par erreur.

```java
private final Object lock = new Object();

void record(BigDecimal orderTotal) {
    // ... code non partagé : validation, calculs locaux (hors verrou)
    synchronized (lock) {
        orderCount++;
        revenue = revenue.add(orderTotal);
    }
}
```

> ❌ **Piège : protéger l'écriture et oublier la lecture.** Si `summary()` n'est pas `synchronized`, elle peut lire `orderCount` déjà incrémenté et `revenue` pas encore mis à jour. Elle peut même lire de vieilles valeurs en cache. **Règle : tous les accès à un état partagé et mutable passent par le même verrou**, lectures comprises.

> ❌ **Piège : `volatile` ne rend pas `count++` atomique.** Le mot-clé `volatile` garantit qu'une écriture est **vue** par les autres threads. Il ne protège pas la séquence lire-modifier-écrire : `volatile int count; count++;` perd toujours des incréments. `volatile` convient à un simple drapeau (`volatile boolean running`), pas à un compteur.

Besoin	Outil
Un compteur, une seule variable	`AtomicInteger`, `AtomicLong`, `LongAdder`
Plusieurs variables à garder cohérentes	`synchronized` (ou `ReentrantLock`)
Une `Map` partagée	`ConcurrentHashMap` et ses opérations atomiques (§1.7)
Un objet qui ne change jamais	Rien : un objet **immuable** (`record`, `BigDecimal`, `String`) se partage sans risque

Le dernier point est essentiel. Les `record` de S2 (`Dish`) sont immuables : vous pouvez les partager entre mille threads sans aucune précaution. L'immuabilité est la meilleure protection contre les race conditions.

1.7 🟢 `ConcurrentHashMap` et ses opérations atomiques

Le problème. Dans le modèle de S2, `DeliveryPlatform` stocke les commandes dans une `HashMap`. Une `HashMap` n'est **pas** conçue pour être modifiée par plusieurs threads : des écritures simultanées peuvent perdre des entrées, voire corrompre sa structure interne.

Comment ? `ConcurrentHashMap` est une `Map` thread-safe : chaque opération **individuelle** (`get`, `put`, `remove`…) est sûre, sans verrou global. Plusieurs threads lisent et écrivent en parallèle.

Mais attention : « thread-safe » ne s'applique qu'à **chaque appel pris isolément**. Une **suite** d'appels n'est pas atomique.

```java
// Fichier : DishPopularity.java
import java.math.BigDecimal;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Executors;

enum DishSize { SMALL, MEDIUM, LARGE }

record Dish(String name, BigDecimal price, DishSize size) {}

void main() {
    Dish pizza = new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM);

    // ❌ ConcurrentHashMap, mais get puis put : deux appels séparés
    Map<Dish, Integer> unsafe = new ConcurrentHashMap<>();
    // ✅ merge : lire, additionner et écrire en une seule opération atomique
    Map<Dish, Integer> safe = new ConcurrentHashMap<>();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int t = 0; t < 1_000; t++) {
            executor.submit(() -> {
                for (int i = 0; i < 100; i++) {
                    int current = unsafe.getOrDefault(pizza, 0); // un autre thread peut écrire ici…
                    unsafe.put(pizza, current + 1);              // … et on écrase sa valeur
                    safe.merge(pizza, 1, Integer::sum);
                }
            });
        }
    }
    IO.println("get puis put : " + unsafe.get(pizza) + " pizzas (attendu : 100000)");
    IO.println("merge        : " + safe.get(pizza) + " pizzas (attendu : 100000)");
}
```

Sortie typique :

```
get puis put : 71066 pizzas (attendu : 100000)
merge        : 100000 pizzas (attendu : 100000)
```

Remarquez que `Dish`, un `record`, sert de clé : son `equals` / `hashCode` (S2) est généré à partir de ses composants. Deux `Dish` identiques comptent pour le même plat.

Les opérations atomiques de `ConcurrentHashMap` à connaître :

Méthode	Ce qu'elle fait, atomiquement	Exemple FoodFast
`merge(k, v, f)`	absent : met `v` ; présent : met `f(ancien, v)`	compter les plats commandés
`putIfAbsent(k, v)`	met `v` seulement si `k` est absent ; renvoie l'ancienne valeur ou `null`	enregistrer une commande sans en écraser une autre
`computeIfAbsent(k, f)`	absent : calcule et met `f(k)`	créer la liste des commandes d'un client
`compute(k, f)`	met `f(k, ancien)` ; si `f` renvoie `null`, supprime	décrémenter un stock (§1.9)
`remove(k)`	supprime et **renvoie** la valeur, ou `null`	consommer un code promo (§1.8)

Méthode	Ce qu'elle fait, atomiquement	Exemple FoodFast
`remove(k, v)` / `replace(k, ancien, nouveau)`	supprime ou remplace seulement si la valeur est encore celle attendue	changer un statut si personne ne l'a changé avant

Règles pour les fonctions passées à `compute`, `merge`… : elles doivent être **courtes**, **sans effet de bord**, et ne **jamais** modifier la map elle-même (d'autres threads attendent pendant leur exécution).

> ❌ **Piège : `null` interdit.** `ConcurrentHashMap` refuse les clés et les valeurs `null` (`NullPointerException`). C'est voulu : `get(k) == null` signifie toujours « absent », jamais « présent avec la valeur `null` ».

`DeliveryPlatform` thread-safe

Voici la version S4 de `DeliveryPlatform`. Deux changements par rapport à S2 : la `HashMap` devient une `ConcurrentHashMap`, et `placeOrder` refuse d'**écraser** une commande existante, grâce à `putIfAbsent`.

```
// Fichier : ThreadSafePlatform.java
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDateTime;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Executors;

// --- Modèle FoodFast (extrait de S2, méthodes inutiles ici retirées) ---
enum DishSize { SMALL, MEDIUM, LARGE }
enum OrderStatus { PENDING, CONFIRMED, PREPARING, IN_DELIVERY, DELIVERED, CANCELLED }

record Dish(String name, BigDecimal price, DishSize size) {
    Dish {
        Objects.requireNonNull(name, "name");
        Objects.requireNonNull(price, "price");
        Objects.requireNonNull(size, "size");
        price = price.setScale(2, RoundingMode.HALF_UP);
    }
}

abstract class User {
    private final UUID id = UUID.randomUUID();
    private final String name;
    private final String email;
    protected User(String name, String email) {
        this.name = Objects.requireNonNull(name);
        this.email = Objects.requireNonNull(email);
    }
    public UUID getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public abstract String role();
}

class Customer extends User {
    private final String address;
    Customer(String name, String email, String address) {
        super(name, email);
        this.address = Objects.requireNonNull(address);
    }
    public String getAddress() { return address; }
    @Override public String role() { return "CLIENT"; }
}

class Order {
    private final String id = "order-" + UUID.randomUUID();
    private final Customer customer;
    private final Map<Dish, Integer> items = new LinkedHashMap<>();
    private final LocalDateTime createdAt = LocalDateTime.now();
    private OrderStatus status = OrderStatus.PENDING;

    Order(Customer customer) { this.customer = Objects.requireNonNull(customer); }

    public void addDish(Dish dish, int quantity) {
        if (status != OrderStatus.PENDING) throw new IllegalStateException("Commande déjà validée");
        if (quantity <= 0) throw new IllegalArgumentException("Quantité invalide : " + quantity);
        items.merge(dish, quantity, Integer::sum);
    }
    public void confirm() {
        if (status != OrderStatus.PENDING || items.isEmpty()) throw new
IllegalStateException("Confirmation impossible");
        status = OrderStatus.CONFIRMED;
```

```
    }
    public String getId() { return id; }
    public Customer getCustomer() { return customer; }
    public OrderStatus getStatus() { return status; }
}

// --- S4 : la plateforme devient thread-safe ---
class DeliveryPlatform {
    private final Map<String, Order> orders = new ConcurrentHashMap<>();

    public void placeOrder(Order order) {
        // ✅ Atomique : jamais deux commandes sous le même identifiant, jamais d'écrasement
silencieux.
        if (orders.putIfAbsent(order.getId(), order) != null) {
            throw new IllegalStateException("Commande déjà enregistrée : " + order.getId());
        }
    }
    public Optional<Order> findOrderById(String id) { return Optional.ofNullable(orders.get(id)); }
    public Collection<Order> getOrders() { return
Collections.unmodifiableCollection(orders.values()); }
}

void main() {
    var platform = new DeliveryPlatform();
    var pizza = new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM);

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int i = 0; i < 10_000; i++) {
            int n = i;
            executor.submit(() -> {
                // Chaque commande est construite par UN seul thread, puis publiée.
                var customer = new Customer("Client " + n, "c" + n + "@example.com", n + " rue de
Paris");
                var order = new Order(customer);
                order.addDish(pizza, 1);
                order.confirm();
                platform.placeOrder(order);
            });
        }
    }
    IO.println("Commandes enregistrées : " + platform.getOrders().size() + " (attendu : 10000)");

    Order first = platform.getOrders().iterator().next();
    try {
        platform.placeOrder(first); // même identifiant : refusé
    } catch (IllegalStateException e) {
        IO.println("Second enregistrement refusé : " + e.getClass().getSimpleName());
    }
}
```

🔵 **Et `Order`, est-il thread-safe ?** Non : `addDish` modifie une `LinkedHashMap` sans verrou. Ce n'est pas un problème tant qu'**une commande est construite par un seul thread** (le thread de la requête du client), puis publiée dans la plateforme. On appelle cela le **confinement** : un objet mutable qui ne quitte pas son thread n'a pas besoin de verrou. Si plusieurs threads devaient modifier la même commande (le livreur et le restaurant changent son statut), il faudrait synchroniser ses méthodes.

1.8 🔒 TOCTOU : « vérifier puis agir »

Le problème. Le **TOCTOU** (*Time Of Check to Time Of Use*, CWE-367) est une race condition qui devient une **faille de sécurité**. Le code **vérifie** une condition, puis **agit** en supposant qu'elle est toujours vraie. Entre les deux, un autre thread (ou un attaquant) a changé la situation.

Exemple FoodFast : le code promo `BIENVENUE10` offre 10 € et ne doit servir **qu'une seule fois**. Un attaquant écrit un script qui envoie la même requête 200 fois en parallèle. C'est une attaque classique sur les sites marchands (*race condition attack* sur les coupons, cartes cadeaux, virements).

```java
// Fichier : PromoCodeRace.java
import java.math.BigDecimal;
import java.time.Duration;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

class InvalidPromoCodeException extends RuntimeException {
    InvalidPromoCodeException() { super("Code promo invalide ou déjà utilisé"); }
}

class PromoCodeService {
    private final Map<String, BigDecimal> unusedCodes = new ConcurrentHashMap<>();

    PromoCodeService() {
        unusedCodes.put("BIENVENUE10", new BigDecimal("10.00"));
    }

    // ❌ TOCTOU : la vérification et l'action sont deux étapes séparées
    BigDecimal redeem(String code) throws InterruptedException {
        BigDecimal amount = unusedCodes.get(code);   // CHECK : le code existe-t-il ?
        if (amount == null) throw new InvalidPromoCodeException();
        saveDiscount(code, amount);                  // fenêtre de course
        unusedCodes.remove(code);                    // USE : on le consomme
        return amount;
    }

    // Simule l'enregistrement de la remise en base : quelques millisecondes.
    private void saveDiscount(String code, BigDecimal amount) throws InterruptedException {
        Thread.sleep(Duration.ofMillis(5));
    }
}

void main() {
    var service = new PromoCodeService();
    var accepted = new AtomicInteger();
    var refused = new AtomicInteger();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int i = 0; i < 200; i++) {                // 200 requêtes simultanées
            executor.submit(() -> {
                try {
                    service.redeem("BIENVENUE10");
                    accepted.incrementAndGet();
                } catch (InvalidPromoCodeException e) {
                    refused.incrementAndGet();
                }
                return null;                           // Callable : sleep peut lever
InterruptedException
            });
        }
    }
    IO.println("Code BIENVENUE10 accepté " + accepted.get() + " fois, refusé " + refused.get() + "
fois");
}
```

```
Code BIENVENUE10 accepté 200 fois, refusé 0 fois
```

Le code à usage unique a été utilisé **200 fois** : 2 000 € de remise au lieu de 10 €.

Notez que la map est **déjà** une `ConcurrentHashMap` : chaque appel est thread-safe, mais la **séquence** `get` → … → `remove` ne l'est pas. Les 200 threads passent tous le `get` avant que le premier n'arrive au `remove`. Le `sleep` de 5 ms élargit la fenêtre pour rendre la démonstration reproductible ; sans lui, la faille existe toujours, elle est seulement plus rare. Et un attaquant sait élargir la fenêtre lui-même, en chargeant le serveur.

La solution : une seule opération atomique qui vérifie ET agit. `ConcurrentHashMap.remove(key)` supprime l'entrée et renvoie l'ancienne valeur, en une seule étape. Un seul thread peut recevoir le montant ; tous les autres reçoivent `null`.

```java
// Fichier : PromoCodeSafe.java
import java.math.BigDecimal;
import java.time.Duration;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

class InvalidPromoCodeException extends RuntimeException {
    InvalidPromoCodeException() { super("Code promo invalide ou déjà utilisé"); }
}

interface PromoCodes {
    BigDecimal redeem(String code) throws InterruptedException;
}

// ✅ Solution 1 (préférée) : l'opération atomique de la map fait la vérification ET l'action.
class AtomicPromoCodes implements PromoCodes {
    private final Map<String, BigDecimal> unusedCodes = new ConcurrentHashMap<>();
    AtomicPromoCodes() { unusedCodes.put("BIENVENUE10", new BigDecimal("10.00")); }

    public BigDecimal redeem(String code) throws InterruptedException {
        BigDecimal amount = unusedCodes.remove(code);  // CHECK + USE en une seule étape
        if (amount == null) throw new InvalidPromoCodeException();
        saveDiscount(code, amount);                    // seul le gagnant arrive ici
        return amount;
    }
    private void saveDiscount(String code, BigDecimal amount) throws InterruptedException {
        Thread.sleep(Duration.ofMillis(5));
    }
}

// ✅ Solution 2 : un verrou qui couvre la vérification ET l'action,
// et que TOUS les accès à unusedCodes prennent.
class LockedPromoCodes implements PromoCodes {
    private final Map<String, BigDecimal> unusedCodes = new HashMap<>(); // protégée par le verrou
    LockedPromoCodes() { unusedCodes.put("BIENVENUE10", new BigDecimal("10.00")); }

    public synchronized BigDecimal redeem(String code) throws InterruptedException {
        BigDecimal amount = unusedCodes.get(code);     // CHECK…
        if (amount == null) throw new InvalidPromoCodeException();
        saveDiscount(code, amount);
        unusedCodes.remove(code);                      // … et USE sous le même verrou
        return amount;
    }
    public synchronized void addCode(String code, BigDecimal amount) {
        unusedCodes.put(code, amount);                 // même verrou pour l'écriture
    }
    private void saveDiscount(String code, BigDecimal amount) throws InterruptedException {
        Thread.sleep(Duration.ofMillis(5));
    }
}

int attack(PromoCodes service) {
    var accepted = new AtomicInteger();
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int i = 0; i < 200; i++) {
            executor.submit(() -> {
                try {
                    service.redeem("BIENVENUE10");
                    accepted.incrementAndGet();
                } catch (InvalidPromoCodeException e) {
```

```
                    // refus attendu pour 199 requêtes sur 200
                }
                return null;
            });
        }
    }
    return accepted.get();
}

void main() {
    IO.println("remove atomique : accepté " + attack(new AtomicPromoCodes()) + " fois sur 200");
    IO.println("verrou          : accepté " + attack(new LockedPromoCodes()) + " fois sur 200");
}
```

```
remove atomique : accepté 1 fois sur 200
verrou          : accepté 1 fois sur 200
```

Comparez les deux solutions :

- **Solution 1** (opération atomique de la map) : aucun verrou à gérer, et les requêtes perdantes échouent immédiatement. C'est la solution à préférer quand l'API le permet.
- **Solution 2** (verrou) : elle fonctionne, mais **toutes** les requêtes attendent leur tour, y compris pendant les 5 ms de `saveDiscount`. Sous forte charge, c'est un goulot d'étranglement. Et elle n'est correcte que si **tous** les accès à `unusedCodes` prennent ce verrou.

> 🔒 **Sécurité — Pourquoi l'ancienne correction était fausse.** L'ancienne version de ce cours « corrigeait » un TOCTOU en ajoutant `synchronized` sur `deleteAllOrders(User user)`, qui vérifiait `user.isAdmin()`. Le verrou pris était celui de la **plateforme**. Le thread qui retirait les droits d'administrateur modifiait `user` **sans prendre ce verrou** : il n'était donc jamais bloqué. Un verrou ne protège que contre les threads qui **prennent le même verrou**. Retenez les deux conditions :
>
> 1. le verrou couvre la vérification **et** l'action ;
> 2. **tous** les codes qui modifient la donnée vérifiée prennent **le même** verrou.
>
> Si vous ne contrôlez pas tous ces codes, préférez une opération atomique sur la donnée elle-même.

Reconnaître un TOCTOU : cherchez un `if` qui teste un état partagé, suivi d'une action qui suppose que le test est toujours vrai.

❌ Vérifier puis agir	✅ Atomique
`if (!map.containsKey(k)) map.put(k, v);`	`map.putIfAbsent(k, v)`
`if (map.containsKey(k)) { v = map.get(k); map.remove(k); }`	`v = map.remove(k)`
`if (stock.get(d) > 0) stock.put(d, stock.get(d) - 1);`	`stock.compute(d, …)` (§1.9)
`if (!Files.exists(p)) Files.writeString(p, s);`	`Files.writeString(p, s, CREATE_NEW)` (§1.11)
`SELECT` du stock, puis `UPDATE` en Java	`UPDATE … SET n = n - 1 WHERE n > 0`, ou contrainte `CHECK` (§2.9)

1.9 🔒 TOCTOU sur le stock : la dernière portion

Même faille, autre cas : il reste **un** tiramisu, et 50 clients cliquent « Commander » en même temps. Le code ❌ `if (portions > 0) { portions = portions − 1; … }` en vend plusieurs. La correction utilise `compute` : la lecture du stock, le test et l'écriture se font **dans** l'opération atomique.

```
// Fichier : LastPortion.java
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

class OutOfStockException extends RuntimeException {
    OutOfStockException(String dish) { super("Plus de portion disponible : " + dish); }
}

class Kitchen {
    private final Map<String, Integer> portions = new ConcurrentHashMap<>();

    void restock(String dish, int quantity) {
        portions.merge(dish, quantity, Integer::sum);
    }

    // ✅ Test et décrément dans la même opération atomique.
    // Lever une exception dans la fonction annule l'opération : la map reste inchangée.
    void reserve(String dish) {
        portions.compute(dish, (d, left) -> {
            if (left == null || left == 0) throw new OutOfStockException(d);
            return left - 1;
        });
    }

    int portionsLeft(String dish) {
        return portions.getOrDefault(dish, 0);
    }
}

void main() {
    var kitchen = new Kitchen();
    kitchen.restock("Tiramisu", 1);
    var sold = new AtomicInteger();
    var refused = new AtomicInteger();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int i = 0; i < 50; i++) {
            executor.submit(() -> {
                try {
                    kitchen.reserve("Tiramisu");
                    sold.incrementAndGet();
                } catch (OutOfStockException e) {
                    refused.incrementAndGet();
                }
            });
        }
    }
    IO.println("Vendus : " + sold.get() + ", refusés : " + refused.get()
            + ", restant : " + kitchen.portionsLeft("Tiramisu"));
}
```

```
Vendus : 1, refusés : 49, restant : 0
```

> 🔵 Dans une vraie application, le stock vit en **base de données**, partagée par plusieurs serveurs. Un verrou Java ne protège alors que les threads d'**un seul** serveur. La vérification doit être atomique **dans la base** : `UPDATE stock SET portions = portions - 1 WHERE dish = ? AND portions > 0`, puis on vérifie le nombre de lignes modifiées (0 signifie « épuisé »). Ou bien une contrainte `CHECK (portions >= 0)` et une transaction (§2.9).

1.10 🔵 Deadlocks

Le problème. Les verrous corrigent les race conditions, mais ils en créent un autre : le **deadlock** (interblocage). Le thread A détient le verrou 1 et attend le verrou 2. Le thread B détient le verrou 2 et attend le verrou 1. Aucun des deux n'avancera jamais. Le programme ne plante pas : il **gèle**.

Exemple FoodFast : la prise de commande verrouille le stock, puis la liste des commandes. L'annulation verrouille la liste des commandes, puis le stock.

```
// Fichier : DeadlockDemo.java
import java.lang.management.ManagementFactory;
import java.util.concurrent.CountDownLatch;

void main() throws InterruptedException {
    Object stockLock = new Object();
    Object ordersLock = new Object();
    var bothHoldFirstLock = new CountDownLatch(2);

    // Threads « démons » : ils n'empêchent pas la JVM de s'arrêter à la fin de main().
    Thread.ofPlatform().daemon(true).name("prise-commande").start(() -> {
        synchronized (stockLock) {
            bothHoldFirstLock.countDown();
            awaitQuietly(bothHoldFirstLock);
            synchronized (ordersLock) { IO.println("commande enregistrée"); }
        }
    });
    Thread.ofPlatform().daemon(true).name("annulation").start(() -> {
        synchronized (ordersLock) {                  // ❌ ordre inverse
            bothHoldFirstLock.countDown();
            awaitQuietly(bothHoldFirstLock);
            synchronized (stockLock) { IO.println("commande annulée"); }
        }
    });

    Thread.sleep(500);
    long[] blocked = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
    IO.println("Threads en deadlock : " + (blocked == null ? 0 : blocked.length));
}

void awaitQuietly(CountDownLatch latch) {
    try {
        latch.await();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
```

```
Threads en deadlock : 2
```

Le `CountDownLatch` garantit ici que chaque thread a pris son premier verrou avant de demander le second : le deadlock se produit à chaque fois. En production, il ne se produit que de temps en temps, ce qui le rend très difficile à

diagnostiquer. Sur une application gelée, la commande `jstack <pid>` affiche « Found one Java-level deadlock ».

La solution : toujours prendre les verrous dans le même ordre.

```
// Fichier : DeadlockFixed.java
import java.util.concurrent.Executors;

void main() {
    Object stockLock = new Object();
    Object ordersLock = new Object();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        executor.submit(() -> {
            synchronized (stockLock) {
                synchronized (ordersLock) { IO.println("commande enregistrée"); }
            }
        });
        executor.submit(() -> {
            synchronized (stockLock) {              // ✅ même ordre : stock, puis commandes
                synchronized (ordersLock) { IO.println("commande annulée"); }
            }
        });
    }
    IO.println("Aucun deadlock");
}
```

Autres règles qui évitent les deadlocks :

- garder les sections `synchronized` **courtes**, et n'y **jamais** faire d'appel réseau ou de requête SQL ;
- ne pas appeler de code « étranger » (une lambda fournie par l'appelant, une méthode redéfinissable) en tenant un verrou ;
- préférer les opérations atomiques (`ConcurrentHashMap`, `AtomicInteger`), qui n'exposent aucun verrou ;
- avec `ReentrantLock`, `tryLock(délai)` permet d'abandonner au lieu d'attendre indéfiniment.

1.11 🔵🔒 TOCTOU sur les fichiers

Le TOCTOU le plus célèbre ne concerne pas une `Map`, mais le **système de fichiers**. Le code vérifie qu'un fichier n'existe pas, puis le crée. Entre les deux, un attaquant local crée à cet emplacement un **lien symbolique** vers un fichier sensible. Le programme écrit alors dans ce fichier.

```
Path receipt = Path.of("/tmp/foodfast/receipt-123.txt");

// ❌ Vérifier puis agir : une fenêtre de course entre exists et writeString
if (!Files.exists(receipt)) {
    Files.writeString(receipt, content);
}

// ✅ Une seule opération : le système crée le fichier, ou échoue
//    (FileAlreadyExistsException) s'il existe déjà, lien symbolique compris.
Files.writeString(receipt, content, StandardOpenOption.CREATE_NEW);
```

Le principe est le même que pour la `Map` : on demande au système d'exploitation de faire la vérification **et** l'action en une seule opération.

1.12 🟣 Scoped values (JEP 506, final en Java 25)

Le problème. Une requête traverse plusieurs couches : contrôleur, service, repository, journal d'audit. Le journal d'audit veut savoir **quel utilisateur** a déclenché l'action. On ne veut pas ajouter un paramètre `User currentUser` à toutes les méthodes. Historiquement, on utilisait un `ThreadLocal` : une variable dont chaque thread a sa propre copie. Mais un `ThreadLocal` est **mutable** (n'importe quel code peut le modifier), sa durée de vie est **floue** (il faut penser à `remove()`, sinon il fuit vers la tâche suivante du pool), et il coûte cher avec des millions de threads virtuels.

Comment ? Une `ScopedValue` (finale en Java 25) est liée à une valeur **pour la durée d'un appel**, puis elle disparaît automatiquement. La liaison est **immuable** : le code appelé peut lire la valeur, pas la remplacer.

```
// Fichier : ScopedValueDemo.java
import java.util.concurrent.Executors;

final class RequestContext {
    static final ScopedValue<String> CURRENT_USER = ScopedValue.newInstance();
    private RequestContext() {}
}

final class AuditLog {
    static void record(String action) {
        // Lit l'utilisateur de la requête en cours, sans qu'on le passe en paramètre.
        String user = RequestContext.CURRENT_USER.orElse("anonyme");
        IO.println("[audit] " + user + " : " + action);
    }
}

final class OrderService {
    void cancel(String orderId) {
        // ... annulation de la commande
        AuditLog.record("annule " + orderId);
    }
}

void main() {
    var service = new OrderService();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        // Deux requêtes simultanées, chacune avec son utilisateur.
        executor.submit(() -> ScopedValue.where(RequestContext.CURRENT_USER, "alice@example.com")
                .run(() -> service.cancel("order-1")));
        executor.submit(() -> ScopedValue.where(RequestContext.CURRENT_USER, "bob@example.com")
                .run(() -> service.cancel("order-2")));
    }

    // Hors de tout where(...) : la valeur n'est pas liée.
    IO.println("Liée dans main ? " + RequestContext.CURRENT_USER.isBound());
    service.cancel("order-3");
}
```

Sortie (l'ordre des deux premières lignes peut varier) :

```
[audit] alice@example.com : annule order-1
[audit] bob@example.com : annule order-2
Liée dans main ? false
[audit] anonyme : annule order-3
```

À retenir :

- `ScopedValue.newInstance()` crée la clé, en général dans un champ `static final` ;
- `ScopedValue.where(clé, valeur).run(...)` (ou `.call(...)` pour renvoyer un résultat) lie la valeur pendant l'appel ;
- `get()` lit la valeur, et lève `NoSuchElementException` si elle n'est pas liée ; `isBound()` et `orElse(...)` permettent de tester.

> 🔒 Une `ScopedValue` est un bon support pour l'**identité de l'utilisateur courant** : le code profond ne peut pas la remplacer par erreur (ou par malveillance), et elle ne « fuit » pas d'une requête à la suivante comme un `ThreadLocal` oublié dans un pool.

1.13 🟣 À venir : la concurrence structurée (preview)

Vous croiserez peut-être `StructuredTaskScope` dans des articles. Cette API (*structured concurrency*, JEP 505) est encore en **preview** en Java 25 : elle ne compile pas sans l'option `--enable-preview`, et son API change d'une version à l'autre. Nous ne l'enseignons donc pas. Son idée mérite d'être connue : lancer plusieurs sous-tâches (appeler le restaurant **et** le service de paiement), et si l'une échoue, annuler automatiquement les autres. C'est aussi par elle que les `ScopedValue` sont transmises aux sous-tâches.

---

2. Persistance avec JDBC

2.1 🟢 Pourquoi une base de données ? JDBC et H2

Le problème. Les commandes de `DeliveryPlatform` vivent dans une `Map` en mémoire : un redémarrage du serveur, et tout est perdu. Et deux serveurs ne partagent pas leur mémoire.

La solution : une **base de données relationnelle**. Java y accède avec **JDBC** (*Java Database Connectivity*), l'API standard du paquet `java.sql`, présente dans le JDK depuis 1997. Tous les outils de plus haut niveau (JPA, Hibernate, Spring Data) reposent sur JDBC.

```
Votre code ──► API JDBC (java.sql : Connection, PreparedStatement, ResultSet)
                  │
                  ▼
            Driver JDBC (un .jar fourni par l'éditeur : H2, PostgreSQL, MySQL…)
                  │
                  ▼
            Base de données
```

Votre code ne dépend que de l'API `java.sql`. Changer de base, c'est changer de driver et d'URL.

Dans ce cours, nous utilisons **H2**, une base écrite en Java qui peut fonctionner **en mémoire** : aucune installation, aucun serveur. Dans un projet Maven, il suffit d'une dépendance :

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <version>2.5.252</version>
</dependency>
```

(Vérifiez la dernière version sur Maven Central. Une dépendance non mise à jour est une faille potentielle.)

L'URL de connexion :

```
jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1
|    |  |   |        └── garder la base tant que la JVM tourne
|    |  |   |            (sinon elle disparaît à la fermeture de la dernière connexion)
|    |  |   └── nom de la base
|    |  └── en mémoire (« file » pour un fichier sur disque)
|    └── driver H2
└── protocole JDBC
```

Pour PostgreSQL, l'URL serait `jdbc:postgresql://localhost:5432/foodfast`. Le reste du code est identique.

2.2 🟢 Se connecter : `Connection` et try-with-resources

Les trois objets JDBC à connaître :

Objet	Rôle	À fermer ?
`Connection`	la connexion à la base (une session réseau, coûteuse)	oui
`PreparedStatement`	une requête SQL **précompilée**, avec des paramètres `?`	oui
`ResultSet`	un curseur sur les lignes renvoyées par un `SELECT`	oui

Les trois sont `AutoCloseable` : on les ouvre **toujours** dans un try-with-resources (S3). Une connexion non fermée est une **fuite** : la base n'accepte qu'un nombre limité de connexions, et l'application finit par ne plus pouvoir s'y connecter du tout.

```java
// Fichier : H2Hello.java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

void main() throws SQLException {
    String url = "jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1";
    // Identifiants lus dans l'environnement, avec une valeur par défaut pour la démo H2 (§2.3).
    String user = System.getenv().getOrDefault("FOODFAST_DB_USER", "sa");
    String password = System.getenv().getOrDefault("FOODFAST_DB_PASSWORD", "");

    try (Connection conn = DriverManager.getConnection(url, user, password);
         PreparedStatement ps = conn.prepareStatement("SELECT H2VERSION()");
         ResultSet rs = ps.executeQuery()) {
        rs.next();
        IO.println("Connecté à H2 " + rs.getString(1));
    } // rs, ps puis conn sont fermés, dans l'ordre inverse de leur ouverture
}
```

`DriverManager` trouve tout seul le driver H2 présent dans le classpath. Il suffit que le `.jar` soit là.

2.3 🔒 Identifiants hors du code et moindre privilège

Le problème. L'ancienne version de ce cours contenait :

```
String password = "secret"; // ❌ mot de passe codé en dur
```

Un mot de passe dans le code source finit **dans Git**, donc dans l'historique, dans chaque clone, chez chaque prestataire, et parfois sur un dépôt public. Des robots parcourent GitHub en permanence à la recherche de ces secrets. Supprimer la ligne dans un commit suivant ne sert à rien : l'historique la conserve. Un secret qui a fuité doit être **changé**.

La solution : le code **lit** les identifiants dans son environnement d'exécution. En développement, ce sont des variables d'environnement (ou un fichier `.env` listé dans `.gitignore`). En production, c'est un gestionnaire de secrets (Vault, AWS Secrets Manager, secrets Kubernetes…), qui les injecte aussi sous forme de variables d'environnement ou de fichiers.

```
// Fichier : DbConfigDemo.java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/** Configuration de la base, lue dans l'environnement. */
record DbConfig(String url, String user, String password) {

    static DbConfig fromEnvironment() {
        return new DbConfig(
                env("FOODFAST_DB_URL", "jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1"),
                env("FOODFAST_DB_USER", "sa"),
                env("FOODFAST_DB_PASSWORD", "")); // défaut acceptable UNIQUEMENT pour H2 en mémoire
    }

    private static String env(String name, String demoDefault) {
        String value = System.getenv(name);
        return value != null ? value : demoDefault;
    }

    Connection connect() throws SQLException {
        return DriverManager.getConnection(url, user, password);
    }

    // 🔒 Le toString() généré par un record afficherait le mot de passe dans les logs (S3).
    @Override
    public String toString() {
        return "DbConfig[url=" + url + ", user=" + user + ", password=***]";
    }
}

void main() throws SQLException {
    var config = DbConfig.fromEnvironment();
    IO.println(config); // peut être journalisé sans risque
    try (Connection conn = config.connect()) {
        IO.println("Connexion ouverte : " + !conn.isClosed());
    }
}
```

> ❌ **Piège : le `toString()` des records.** Un `record` génère un `toString()` qui affiche **tous** ses composants. `log.log(INFO, "Config : " + config)` écrirait le mot de passe en clair dans les journaux. Redéfinissez `toString()` pour masquer les secrets.

> 🔒 **Sécurité — En production, pas de valeur par défaut.** La valeur par défaut `""` n'a de sens que pour une base H2 en mémoire, jetable. En production, si la variable est absente, l'application doit **refuser de démarrer** avec un message clair (`IllegalStateException("Variable FOODFAST_DB_PASSWORD absente")`), plutôt que de tenter une connexion sans mot de passe.

Le moindre privilège. Le compte utilisé par l'application ne doit avoir que les droits **nécessaires** : lire et insérer des commandes, pas supprimer des tables ni créer des utilisateurs. Si une injection SQL (§2.7) passe malgré tout, les dégâts sont limités à ce que ce compte peut faire. Le compte administrateur sert uniquement à créer le schéma, au déploiement.

```
// Fichier : LeastPrivilege.java
import java.security.SecureRandom;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Base64;

void main() throws SQLException {
    String adminUrl = "jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1";
    String appUrl = "jdbc:h2:mem:foodfast"; // même base ; seul l'administrateur règle DB_CLOSE_DELAY

    // Le mot de passe du compte applicatif est généré aléatoirement, jamais écrit dans le code.
    byte[] random = new byte[24];
    new SecureRandom().nextBytes(random);
    String appPassword = Base64.getEncoder().encodeToString(random);

    // 1. Au déploiement, l'administrateur crée le schéma et un compte aux droits limités.
    try (Connection admin = DriverManager.getConnection(adminUrl,
            System.getenv().getOrDefault("FOODFAST_DB_USER", "sa"),
            System.getenv().getOrDefault("FOODFAST_DB_PASSWORD", ""));
         Statement st = admin.createStatement()) {
        st.execute("CREATE TABLE orders (id VARCHAR(64) PRIMARY KEY, customer_name VARCHAR(100))");
        try (PreparedStatement ps = admin.prepareStatement("CREATE USER foodfast_app PASSWORD ?")) {
            ps.setString(1, appPassword);
            ps.execute();
        }
        st.execute("GRANT SELECT, INSERT ON orders TO foodfast_app"); // ni UPDATE, ni DELETE, ni
DROP
    }

    // 2. L'application se connecte avec ce compte.
    try (Connection app = DriverManager.getConnection(appUrl, "foodfast_app", appPassword);
         Statement st = app.createStatement()) {
        st.executeUpdate("INSERT INTO orders VALUES ('order-1', 'Alice')");
        IO.println("INSERT autorisé");
        try {
            st.execute("DROP TABLE orders");
        } catch (SQLException e) {
            IO.println("DROP TABLE refusé (code " + e.getErrorCode() + ")");
        }
    }
}
```

```
INSERT autorisé
DROP TABLE refusé (code 90096)
```

2.4 🟢 Créer le schéma : du SQL lisible avec les text blocks

Le SQL concaténé ligne à ligne (`"SELECT ... " + "FROM ... "`) est illisible et source d'erreurs (l'espace oublié en fin de ligne). Les **text blocks** (Java 15) permettent d'écrire le SQL comme dans un éditeur SQL :

```
private static final String CREATE_TABLE = """
        CREATE TABLE orders (
            id            VARCHAR(64)   PRIMARY KEY,
            customer_name VARCHAR(100)  NOT NULL,
            total         DECIMAL(10,2) NOT NULL,
            status        VARCHAR(20)   NOT NULL,
            created_at    TIMESTAMP     NOT NULL
        )
        """;
```

L'indentation commune est retirée automatiquement : la position des `"""` fermants fixe la marge.

Correspondance des types entre Java et SQL :

Java	SQL	Méthodes JDBC
`String`	`VARCHAR(n)`	`setString` / `getString`
`BigDecimal` (montants !)	`DECIMAL(p, s)`	`setBigDecimal` / `getBigDecimal`
`int`	`INTEGER`	`setInt` / `getInt`
`LocalDateTime`	`TIMESTAMP`	`setObject` / `getObject(col, LocalDateTime.class)`
`enum`	`VARCHAR`	`setString(status.name())` / `OrderStatus.valueOf(rs.getString(...))`

> ❌ **Piège : `DOUBLE` pour un prix.** Comme en Java (S2), un montant se stocke en `DECIMAL` , jamais en `DOUBLE` ou `FLOAT` .

> ❌ **Piège : un text block n'est pas un `PreparedStatement` .** Un text block est une simple `String` . Y insérer une valeur utilisateur avec `formatted(...)` ou `+` crée exactement la même injection SQL qu'une concaténation (§2.7). Les valeurs passent toujours par des `?` .

2.5 🟢 `PreparedStatement` : écrire une commande

Le code JDBC est regroupé dans une classe **repository** : c'est la seule classe qui connaît le SQL. Le reste de l'application manipule des objets.

Pour écrire, trois étapes :

1. `conn.prepareStatement(sql)` avec un `?` à la place de chaque valeur ;
2. `ps.setXxx(position, valeur)` pour chaque `?` . Les positions commencent à **1** ;
3. `ps.executeUpdate()` , qui renvoie le nombre de lignes modifiées.

```
void save(Order order) throws SQLException {
    try (Connection conn = connect();
         PreparedStatement ps = conn.prepareStatement(INSERT)) {
        ps.setString(1, order.getId());                 // 1er ?
        ps.setString(2, order.getCustomer().getName()); // 2e ?
        ps.setBigDecimal(3, order.getTotal());
        ps.setString(4, order.getStatus().name());      // enum stockée par son nom
        ps.setObject(5, order.getCreatedAt());          // LocalDateTime directement (JDBC 4.2)
        ps.executeUpdate();
    }
}
```

❌ **Piège : les index commencent à 1.** `ps.setString(0, …)` lève une `SQLException`. Les colonnes d'un `ResultSet` sont aussi numérotées à partir de 1.

2.6 🟢 `ResultSet` : relire et mapper vers des objets

Pour lire :

- `ps.executeQuery()` renvoie un `ResultSet`. C'est un **curseur** placé **avant** la première ligne ;
- `rs.next()` avance d'une ligne et renvoie `false` quand il n'y en a plus ;
- `rs.getXxx("colonne")` lit une colonne de la ligne courante.

On transforme ensuite chaque ligne en objet Java : c'est le **mapping**. Comme `Order` génère son identifiant et sa date à la construction (modèle S2), on relit les commandes sous la forme d'un `record OrderSummary` : une vue immuable de ce qui est stocké.

Voici le repository complet, avec une démonstration :

```java
// Fichier : OrderRepositoryDemo.java
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.*;

// --- Modèle FoodFast (extrait de S2, méthodes inutiles ici retirées) ---
enum DishSize { SMALL, MEDIUM, LARGE }
enum OrderStatus { PENDING, CONFIRMED, PREPARING, IN_DELIVERY, DELIVERED, CANCELLED }

record Dish(String name, BigDecimal price, DishSize size) {
    Dish {
        Objects.requireNonNull(name, "name");
        Objects.requireNonNull(price, "price");
        Objects.requireNonNull(size, "size");
        price = price.setScale(2, RoundingMode.HALF_UP);
    }
}

abstract class User {
    private final UUID id = UUID.randomUUID();
    private final String name;
    private final String email;
    protected User(String name, String email) {
        this.name = Objects.requireNonNull(name);
        this.email = Objects.requireNonNull(email);
    }
    public UUID getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public abstract String role();
}

class Customer extends User {
    private final String address;
    Customer(String name, String email, String address) {
        super(name, email);
        this.address = Objects.requireNonNull(address);
    }
    public String getAddress() { return address; }
    @Override public String role() { return "CLIENT"; }
}

class Order {
    private final String id = "order-" + UUID.randomUUID();
    private final Customer customer;
    private final Map<Dish, Integer> items = new LinkedHashMap<>();
    private final LocalDateTime createdAt = LocalDateTime.now();
    private OrderStatus status = OrderStatus.PENDING;

    Order(Customer customer) { this.customer = Objects.requireNonNull(customer); }

    public void addDish(Dish dish, int quantity) {
        if (status != OrderStatus.PENDING) throw new IllegalStateException("Commande déjà validée");
        if (quantity <= 0) throw new IllegalArgumentException("Quantité invalide : " + quantity);
        items.merge(dish, quantity, Integer::sum);
    }
    public BigDecimal getTotal() {
        BigDecimal total = BigDecimal.ZERO;
        for (Map.Entry<Dish, Integer> e : items.entrySet()) {
            total = total.add(e.getKey().price().multiply(BigDecimal.valueOf(e.getValue())));
        }
```

```
        return total;
    }
    public void confirm() {
        if (status != OrderStatus.PENDING || items.isEmpty()) throw new
IllegalStateException("Confirmation impossible");
        status = OrderStatus.CONFIRMED;
    }
    public String getId() { return id; }
    public Customer getCustomer() { return customer; }
    public OrderStatus getStatus() { return status; }
    public LocalDateTime getCreatedAt() { return createdAt; }
}

// --- S3 : exception métier ---
class OrderNotFoundException extends RuntimeException {
    private final String orderId;

    OrderNotFoundException(String orderId) {
        super("Commande introuvable : " + orderId);
        this.orderId = orderId;
    }

    public String getOrderId() { return orderId; }
}

// --- S4 : persistance ---

/** Ce que la base renvoie pour une commande : une vue immuable. */
record OrderSummary(String id, String customerName, BigDecimal total,
                    OrderStatus status, LocalDateTime createdAt) {}

class OrderRepository {
    private static final String CREATE_TABLE = """
            CREATE TABLE IF NOT EXISTS orders (
                id            VARCHAR(64)   PRIMARY KEY,
                customer_name VARCHAR(100)  NOT NULL,
                total         DECIMAL(10,2) NOT NULL,
                status        VARCHAR(20)   NOT NULL,
                created_at    TIMESTAMP     NOT NULL
            )
            """;
    private static final String INSERT = """
            INSERT INTO orders (id, customer_name, total, status, created_at)
            VALUES (?, ?, ?, ?, ?)
            """;
    private static final String SELECT_BY_ID = """
            SELECT id, customer_name, total, status, created_at
            FROM orders
            WHERE id = ?
            """;
    private static final String SELECT_BY_CUSTOMER = """
            SELECT id, customer_name, total, status, created_at
            FROM orders
            WHERE customer_name = ?
            ORDER BY created_at
            """;

    private final String url;
    private final String user;
    private final String password;

    OrderRepository(String url, String user, String password) {
        this.url = url;
```

```
        this.user = user;
        this.password = password;
    }

    private Connection connect() throws SQLException {
        return DriverManager.getConnection(url, user, password);
    }

    void createSchema() throws SQLException {
        try (Connection conn = connect();
             Statement st = conn.createStatement()) {
            st.execute(CREATE_TABLE); // aucune donnée externe : un Statement simple suffit
        }
    }

    void save(Order order) throws SQLException {
        try (Connection conn = connect();
             PreparedStatement ps = conn.prepareStatement(INSERT)) {
            ps.setString(1, order.getId());                 // 1er ?
            ps.setString(2, order.getCustomer().getName()); // 2e ?
            ps.setBigDecimal(3, order.getTotal());
            ps.setString(4, order.getStatus().name());      // enum stockée par son nom
            ps.setObject(5, order.getCreatedAt());          // LocalDateTime directement (JDBC 4.2)
            ps.executeUpdate();
        }
    }

    Optional<OrderSummary> findById(String id) throws SQLException {
        try (Connection conn = connect();
             PreparedStatement ps = conn.prepareStatement(SELECT_BY_ID)) {
            ps.setString(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? Optional.of(toSummary(rs)) : Optional.empty();
            }
        }
    }

    List<OrderSummary> findByCustomerName(String customerName) throws SQLException {
        try (Connection conn = connect();
             PreparedStatement ps = conn.prepareStatement(SELECT_BY_CUSTOMER)) {
            ps.setString(1, customerName);
            try (ResultSet rs = ps.executeQuery()) {
                List<OrderSummary> result = new ArrayList<>();
                while (rs.next()) {
                    result.add(toSummary(rs));
                }
                return result;
            }
        }
    }

    // Une ligne du ResultSet → un objet Java.
    private static OrderSummary toSummary(ResultSet rs) throws SQLException {
        return new OrderSummary(
                rs.getString("id"),
                rs.getString("customer_name"),
                rs.getBigDecimal("total"),
                OrderStatus.valueOf(rs.getString("status")),
                rs.getObject("created_at", LocalDateTime.class));
    }
}

void main() throws SQLException {
```

```
    var repository = new OrderRepository(
            "jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1",
            System.getenv().getOrDefault("FOODFAST_DB_USER", "sa"),
            System.getenv().getOrDefault("FOODFAST_DB_PASSWORD", ""));
    repository.createSchema();

    var pizza = new Dish("Margherita", new BigDecimal("12.5"), DishSize.MEDIUM);
    var tiramisu = new Dish("Tiramisu", new BigDecimal("6.90"), DishSize.SMALL);
    var alice = new Customer("Alice", "alice@example.com", "10 rue de Paris");

    var order = new Order(alice);
    order.addDish(pizza, 2);
    order.addDish(tiramisu, 1);
    order.confirm();
    repository.save(order);

    OrderSummary found = repository.findById(order.getId())
            .orElseThrow(() -> new OrderNotFoundException(order.getId()));
    IO.println("Relue : " + found.customerName() + ", " + found.total() + " €, " + found.status());
    IO.println("Commandes d'Alice : " + repository.findByCustomerName("Alice").size());
    IO.println("Commande inconnue : " + repository.findById("order-inconnue"));
}
```

```
Relue : Alice, 31.90 €, CONFIRMED
Commandes d'Alice : 1
Commande inconnue : Optional.empty
```

Remarques :

- **Une connexion par opération**, ouverte et fermée dans la méthode. Avec `DriverManager`, c'est coûteux (une connexion réseau à chaque appel). En production, on utilise un **pool de connexions** (HikariCP) exposé sous la forme d'un `javax.sql.DataSource` : `dataSource.getConnection()` emprunte une connexion déjà ouverte, et `close()` la rend au pool. Le code du repository ne change presque pas.
- **`findById` renvoie un `Optional`** (S2) : « pas trouvé » n'est pas une erreur technique. C'est l'appelant qui décide si c'est une erreur métier (`OrderNotFoundException`, S3).
- **`SQLException` est une exception vérifiée.** Ici, on la laisse remonter (`throws SQLException`). Dans une application en couches, on l'enveloppe souvent dans une exception non vérifiée propre à la couche de persistance, **en conservant la cause** (`new DataAccessException("…", e)`). Et on ne renvoie jamais son message brut à l'utilisateur : il contient le SQL et des détails du schéma (CWE-209, S3).
- **`SELECT *` est évité** : on liste les colonnes. Le code ne casse pas si une colonne est ajoutée, et on ne lit pas de données inutiles (ou sensibles).

> ❌ **Piège : oublier `rs.next()`.** Le curseur commence **avant** la première ligne. Appeler `rs.getString("id")` sans `rs.next()` lève une `SQLException`.

2.7 🔒 Injection SQL

Le problème. L'injection SQL (CWE-89, catégorie A03 « Injection » du Top 10 OWASP) reste l'une des failles les plus exploitées. Le principe : une **donnée** fournie par l'utilisateur est collée dans le texte d'une requête SQL. Si elle contient des caractères spéciaux du SQL (`'`, `--`, `;`), elle **change la structure** de la requête.

Démonstration complète : le support client recherche les commandes d'un client par son nom.

```java
// Fichier : SqlInjectionDemo.java
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

// ❌ VULNÉRABLE : la saisie de l'utilisateur devient du code SQL.
List<String> findByCustomerUnsafe(Connection conn, String name) throws SQLException {
    String sql = "SELECT id, customer_name, total FROM orders WHERE customer_name = '" + name + "'";
    IO.println("   SQL exécuté : " + sql);
    try (Statement st = conn.createStatement();
         ResultSet rs = st.executeQuery(sql)) {
        return readRows(rs);
    }
}

// ✅ SÛR : la requête a une structure fixe, la saisie n'est qu'une valeur.
List<String> findByCustomer(Connection conn, String name) throws SQLException {
    String sql = "SELECT id, customer_name, total FROM orders WHERE customer_name = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, name);
        try (ResultSet rs = ps.executeQuery()) {
            return readRows(rs);
        }
    }
}

List<String> readRows(ResultSet rs) throws SQLException {
    List<String> rows = new ArrayList<>();
    while (rs.next()) {
        rows.add(rs.getString("id") + " (" + rs.getString("customer_name") + ", "
                + rs.getBigDecimal("total") + " €)");
    }
    return rows;
}

void main() throws SQLException {
    try (Connection conn = DriverManager.getConnection("jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1",
            System.getenv().getOrDefault("FOODFAST_DB_USER", "sa"),
            System.getenv().getOrDefault("FOODFAST_DB_PASSWORD", ""));
         Statement st = conn.createStatement()) {
        st.execute("""
                CREATE TABLE orders (
                    id            VARCHAR(64)   PRIMARY KEY,
                    customer_name VARCHAR(100)  NOT NULL,
                    total         DECIMAL(10,2) NOT NULL
                )""");
        st.execute("""
                INSERT INTO orders VALUES
                    ('order-1', 'Alice', 31.90),
                    ('order-2', 'Bob',   18.00),
                    ('order-3', 'Chloé', 42.50)""");

        String attack = "' OR '1'='1";

        IO.println("❌ Recherche normale « Alice » :");
        IO.println("   " + findByCustomerUnsafe(conn, "Alice"));
        IO.println("❌ Recherche malveillante « " + attack + " » :");
        IO.println("   " + findByCustomerUnsafe(conn, attack));
        IO.println("✅ PreparedStatement, même saisie :");
        IO.println("   " + findByCustomer(conn, attack));
    }
}
```

```
❌ Recherche normale « Alice » :
   SQL exécuté : SELECT id, customer_name, total FROM orders WHERE customer_name = 'Alice'
   [order-1 (Alice, 31.90 €)]
❌ Recherche malveillante « ' OR '1'='1 » :
   SQL exécuté : SELECT id, customer_name, total FROM orders WHERE customer_name = '' OR '1'='1'
   [order-1 (Alice, 31.90 €), order-2 (Bob, 18.00 €), order-3 (Chloé, 42.50 €)]
✅ PreparedStatement, même saisie :
   []
```

Pourquoi ça marche ? Lisez la requête générée : `WHERE customer_name = '' OR '1'='1'`. La première apostrophe de la saisie **ferme** la chaîne SQL ; la suite, `OR '1'='1'`, est interprétée comme du **code**. La condition est vraie pour toutes les lignes : l'attaquant voit les commandes de **tous** les clients.

Avec d'autres saisies, l'attaquant peut :

- contourner une authentification : `admin' --` (le `--` met en commentaire la vérification du mot de passe) ;
- lire d'autres tables : `' UNION SELECT login, password_hash, 0 FROM users --` ;
- modifier ou détruire des données, selon les droits du compte (d'où le moindre privilège, §2.3).

Pourquoi `PreparedStatement` protège ? La requête `… WHERE customer_name = ?` est envoyée et **analysée par la base avant** que la valeur ne soit connue. Sa structure est figée. La valeur est ensuite transmise **séparément**, comme une donnée : la base cherche un client qui s'appelle littéralement `' OR '1'='1`. Il n'y a pas d'« échappement » des apostrophes : la valeur n'est simplement jamais interprétée comme du SQL.

> ❌ **Piège : « j'échappe les apostrophes moi-même ».** Remplacer `'` par `''` à la main, filtrer les mots `OR` ou `DROP` … Ces filtres maison sont contournés (encodages, commentaires, autres caractères spéciaux selon la base). **La seule protection fiable, ce sont les paramètres `?`.** La validation des entrées (S1) reste utile, en complément.

> ❌ **Piège : « cette valeur ne vient pas de l'utilisateur ».** Un identifiant lu dans la base, un en-tête HTTP, un nom de fichier, un champ caché d'un formulaire : tout ce qui vient de l'extérieur du programme peut être contrôlé par un attaquant. Utilisez **toujours** `PreparedStatement` pour les valeurs, même quand elles semblent sûres.

2.8 🔒 Les limites de `PreparedStatement` : les noms de colonnes

Un `?` remplace une **valeur**, jamais un **nom** de colonne ou de table, ni un mot-clé SQL. `ORDER BY ?` ne fait pas ce que vous croyez (la base trie sur une constante). Pour laisser l'utilisateur choisir le tri, on utilise une **liste blanche** : la saisie sert à **choisir** parmi des valeurs connues, elle n'est jamais recopiée dans le SQL.

```java
// Fichier : SortWhitelist.java
import java.sql.*;

// ✅ Liste blanche : l'entrée choisit parmi des colonnes connues, elle n'est jamais copiée dans le SQL.
String sortColumn(String userChoice) {
    return switch (userChoice) {
        case "date" -> "created_at";
        case "total" -> "total";
        default -> throw new IllegalArgumentException("Tri non autorisé");
    };
}

void main() throws SQLException {
    try (Connection conn = DriverManager.getConnection("jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1",
            System.getenv().getOrDefault("FOODFAST_DB_USER", "sa"),
            System.getenv().getOrDefault("FOODFAST_DB_PASSWORD", "")));
         Statement st = conn.createStatement()) {
        st.execute("CREATE TABLE orders (id VARCHAR(64), total DECIMAL(10,2), created_at TIMESTAMP)");
        st.execute("""
                INSERT INTO orders VALUES
                    ('order-1', 31.90, TIMESTAMP '2026-10-23 19:55:00'),
                    ('order-2', 18.00, TIMESTAMP '2026-10-23 20:05:00')""");

        String sql = "SELECT id FROM orders ORDER BY " + sortColumn("total"); // constante sûre
        try (PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) IO.println(rs.getString("id"));
        }

        try {
            sortColumn("total; DROP TABLE orders");
        } catch (IllegalArgumentException e) {
            IO.println("Refusé : " + e.getMessage());
        }
    }
}
```

```
order-2
order-1
Refusé : Tri non autorisé
```

2.9 🔵 Transactions

Le problème. Passer une commande, c'est **deux** écritures : insérer la commande, et décrémenter le stock. Si la seconde échoue (plus de stock), la première ne doit pas rester. Sinon la base contient une commande pour un plat qui n'a jamais été réservé.

Comment ? Par défaut, JDBC est en mode **auto-commit** : chaque requête est validée immédiatement, toute seule. Pour grouper plusieurs requêtes en une **transaction** (tout ou rien) :

1. `conn.setAutoCommit(false)` ;
2. exécuter les requêtes ;
3. `conn.commit()` si tout s'est bien passé, `conn.rollback()` sinon.

```java
// Fichier : TransactionDemo.java
import java.sql.*;

void placeOrder(Connection conn, String orderId, String dish) throws SQLException {
    conn.setAutoCommit(false);
    try (PreparedStatement insert = conn.prepareStatement(
                "INSERT INTO orders (id, dish) VALUES (?, ?)");
         PreparedStatement decrement = conn.prepareStatement(
                "UPDATE stock SET portions = portions - 1 WHERE dish = ?")) {
        insert.setString(1, orderId);
        insert.setString(2, dish);
        insert.executeUpdate();

        decrement.setString(1, dish);
        decrement.executeUpdate();  // viole CHECK (portions >= 0) s'il n'en reste plus

        conn.commit();              // ✅ les deux écritures, ou…
    } catch (SQLException e) {
        conn.rollback();            // ✅ … aucune
        throw e;                    // on n'avale pas l'exception (S3)
    } finally {
        conn.setAutoCommit(true);
    }
}

void main() throws SQLException {
    try (Connection conn = DriverManager.getConnection("jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1",
            System.getenv().getOrDefault("FOODFAST_DB_USER", "sa"),
            System.getenv().getOrDefault("FOODFAST_DB_PASSWORD", ""));
         Statement st = conn.createStatement()) {
        st.execute("CREATE TABLE orders (id VARCHAR(64) PRIMARY KEY, dish VARCHAR(100))");
        st.execute("""
                CREATE TABLE stock (
                    dish     VARCHAR(100) PRIMARY KEY,
                    portions INT NOT NULL CHECK (portions >= 0)
                )""");
        st.execute("INSERT INTO stock VALUES ('Tiramisu', 1)");

        placeOrder(conn, "order-1", "Tiramisu");
        IO.println("order-1 : commande passée");
        try {
            placeOrder(conn, "order-2", "Tiramisu");
        } catch (SQLException e) {
            IO.println("order-2 : refusée, transaction annulée");
        }

        try (ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM orders")) {
            rs.next();
            IO.println("Commandes en base : " + rs.getInt(1)); // 1 : order-2 n'a laissé aucune trace
        }
    }
}
```

```
order-1 : commande passée
order-2 : refusée, transaction annulée
Commandes en base : 1
```

Notez que la contrainte `CHECK (portions >= 0)` règle aussi le TOCTOU de la « dernière portion » (§1.9) **au niveau de la base** : même avec dix serveurs en parallèle, le stock ne peut pas devenir négatif. Les garanties de la base (contraintes, clés primaires, transactions) sont le dernier rempart contre les race conditions entre plusieurs machines.

> 🔵 Dans une vraie application, la connexion vient d'un pool et la transaction couvre souvent plusieurs repositories. Les frameworks (Spring, Jakarta EE) gèrent alors `commit` et `rollback` pour vous (`@Transactional`). Le principe reste exactement celui-ci.

---

3. 🔒 Sécurité : pour aller plus loin

3.1 🔵🔒 Stocker des mots de passe : PBKDF2

Le problème. FoodFast stocke les comptes clients. Si la base fuite (injection SQL, sauvegarde mal protégée, employé malveillant), que trouve l'attaquant ?

Stockage	Ce que l'attaquant obtient
❌ En clair	Tous les mots de passe, immédiatement. Et comme les gens les réutilisent, leurs autres comptes.
❌ Chiffré (AES)	La clé est sur le même serveur : il la vole aussi.
❌ Haché avec SHA-256	Un GPU calcule des milliards de SHA-256 par seconde : les mots de passe courants tombent en quelques minutes. Deux clients au même mot de passe ont le même haché.
✅ Haché **lentement**, avec un **sel** aléatoire	Chaque essai coûte cher, et chaque compte doit être attaqué séparément.

Comment ? Une fonction de hachage de mots de passe est **volontairement lente** et **salée** :

- le **sel** est une valeur aléatoire, différente pour chaque compte, stockée avec le haché. Deux mots de passe identiques donnent deux hachés différents, et les tables précalculées (*rainbow tables*) deviennent inutiles ;
- le **facteur de coût** (ici, le nombre d'itérations) rend chaque calcul lent : environ 0,3 s pour le serveur à chaque connexion (acceptable), mais aussi 0,3 s pour chaque essai de l'attaquant (rédhibitoire).

Le JDK fournit **PBKDF2** (`PBKDF2WithHmacSHA256`). Les recommandations actuelles (OWASP) préfèrent **Argon2id**, ou à défaut bcrypt ou scrypt, disponibles via des bibliothèques (Bouncy Castle, Spring Security). PBKDF2 reste acceptable avec un nombre d'itérations élevé : OWASP recommande **600 000** pour PBKDF2-HMAC-SHA256.

```java
// Fichier : PasswordHashingDemo.java
import java.security.GeneralSecurityException;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;

final class PasswordHasher {
    private static final String ALGORITHM = "PBKDF2WithHmacSHA256";
    private static final int ITERATIONS = 600_000;   // recommandation OWASP pour PBKDF2-HMAC-SHA256
    private static final int SALT_BYTES = 16;
    private static final int HASH_BITS = 256;
    private static final SecureRandom RANDOM = new SecureRandom(); // jamais java.util.Random

    private PasswordHasher() {}

    /** Renvoie « itérations:sel:haché », à stocker tel quel en base. */
    static String hash(char[] password) {
        byte[] salt = new byte[SALT_BYTES];
        RANDOM.nextBytes(salt);                      // un sel aléatoire par compte
        byte[] hash = pbkdf2(password, salt, ITERATIONS);
        return ITERATIONS + ":" + encode(salt) + ":" + encode(hash);
    }

    static boolean verify(char[] password, String stored) {
        String[] parts = stored.split(":");
        int iterations = Integer.parseInt(parts[0]);
        byte[] salt = Base64.getDecoder().decode(parts[1]);
        byte[] expected = Base64.getDecoder().decode(parts[2]);
        byte[] actual = pbkdf2(password, salt, iterations);
        return MessageDigest.isEqual(expected, actual); // ✅ comparaison en temps constant
    }

    private static byte[] pbkdf2(char[] password, byte[] salt, int iterations) {
        PBEKeySpec spec = new PBEKeySpec(password, salt, iterations, HASH_BITS);
        try {
            return SecretKeyFactory.getInstance(ALGORITHM).generateSecret(spec).getEncoded();
        } catch (GeneralSecurityException e) {
            throw new IllegalStateException("PBKDF2 indisponible", e);
        } finally {
            spec.clearPassword();                    // efface la copie du mot de passe
        }
    }

    private static String encode(byte[] bytes) {
        return Base64.getEncoder().encodeToString(bytes);
    }
}

void main() {
    String alice = PasswordHasher.hash("Tiramisu-2026!".toCharArray());
    String bob = PasswordHasher.hash("Tiramisu-2026!".toCharArray()); // même mot de passe…

    IO.println("Stocké pour Alice : " + alice.substring(0, 40) + "…");
    IO.println("Même haché pour Bob ? " + alice.equals(bob));       // … hachés différents
    IO.println("Bon mot de passe : " + PasswordHasher.verify("Tiramisu-2026!".toCharArray(), alice));
    IO.println("Mauvais          : " + PasswordHasher.verify("tiramisu-2026!".toCharArray(), alice));
}
```

Sortie (le sel, donc le haché, change à chaque exécution) :

```
Stocké pour Alice : 600000:8T/8i17lBUgSOAlR2uzjgQ==:8Jo4l+vj…
Même haché pour Bob ? false
Bon mot de passe : true
Mauvais          : false
```

Les détails qui comptent :

- `SecureRandom`, jamais `java.util.Random` (prévisible) pour un sel, un jeton ou une clé.
- `MessageDigest.isEqual` compare en **temps constant**. `Arrays.equals` s'arrête au premier octet différent : en mesurant le temps de réponse, un attaquant peut en déduire combien d'octets sont justes (*timing attack*).
- `char[]` **plutôt que** `String` pour le mot de passe : un tableau peut être effacé après usage (`clearPassword()`), une `String` reste en mémoire jusqu'au passage du ramasse-miettes.
- Les **itérations sont stockées** avec le haché : le jour où vous les augmentez, les anciens comptes restent vérifiables, et on les re-hache à leur prochaine connexion.

3.2 🟣🔒 L'API KDF (JEP 510, finale en Java 25) : dériver des clés, pas hacher des mots de passe

Java 25 ajoute une API standard de **fonctions de dérivation de clés** : la classe `javax.crypto.KDF`, avec l'algorithme **HKDF** (RFC 5869). Son nom ressemble à PBKDF2, mais son usage est **différent**.

À quoi sert HKDF ? À partir d'un secret **déjà fort** (de haute entropie), comme le secret partagé obtenu par un échange de clés Diffie-Hellman ou une clé maîtresse aléatoire, HKDF fabrique **plusieurs clés indépendantes**, une par usage : une clé pour chiffrer, une autre pour signer, etc. C'est ce que fait TLS 1.3 à chaque connexion HTTPS.

```java
// Fichier : HkdfDemo.java
import java.security.SecureRandom;
import java.util.HexFormat;
import javax.crypto.KDF;
import javax.crypto.SecretKey;
import javax.crypto.spec.HKDFParameterSpec;

void main() throws Exception {
    // Secret maître de haute entropie : 32 octets aléatoires (jamais un mot de passe).
    byte[] masterSecret = new byte[32];
    new SecureRandom().nextBytes(masterSecret);
    byte[] salt = "foodfast-v1".getBytes();

    KDF hkdf = KDF.getInstance("HKDF-SHA256");

    // Deux clés indépendantes, dérivées du même secret, séparées par leur « info ».
    SecretKey encryptionKey = hkdf.deriveKey("AES", HKDFParameterSpec.ofExtract()
            .addIKM(masterSecret)
            .addSalt(salt)
            .thenExpand("chiffrement-adresses".getBytes(), 32));
    SecretKey macKey = hkdf.deriveKey("HmacSHA256", HKDFParameterSpec.ofExtract()
            .addIKM(masterSecret)
            .addSalt(salt)
            .thenExpand("signature-jetons".getBytes(), 32));

    IO.println(encryptionKey.getAlgorithm() + " : " + encryptionKey.getEncoded().length * 8 + "
bits");
    IO.println(macKey.getAlgorithm() + " : " + macKey.getEncoded().length * 8 + " bits");
    IO.println("Clés différentes ? " + !HexFormat.of().formatHex(encryptionKey.getEncoded())
            .equals(HexFormat.of().formatHex(macKey.getEncoded())));

    // HKDF est conçu pour être RAPIDE : c'est exactement ce qu'on ne veut pas pour un mot de passe.
    long start = System.nanoTime();
    for (int i = 0; i < 10_000; i++) {
        hkdf.deriveData(HKDFParameterSpec.ofExtract()
                .addIKM(("motdepasse" + i).getBytes())
                .thenExpand(new byte[0], 32));
    }
    IO.println("10 000 dérivations HKDF en " + (System.nanoTime() - start) / 1_000_000 + " ms");
}
```

Sortie typique :

```
AES : 256 bits
HmacSHA256 : 256 bits
Clés différentes ? true
10 000 dérivations HKDF en 53 ms
```

L'API en trois étapes :

1. `KDF.getInstance("HKDF-SHA256")` choisit l'algorithme (`HKDF-SHA384` et `HKDF-SHA512` existent aussi) ;
2. `HKDFParameterSpec.ofExtract().addIKM(secret).addSalt(sel).thenExpand(info, longueur)` décrit la dérivation (*extract* puis *expand*, comme dans la RFC) ;
3. `deriveKey("AES", params)` renvoie une `SecretKey`, et `deriveData(params)` renvoie des octets bruts.

> 🔒 **Sécurité — HKDF ne sert PAS à hacher des mots de passe.** HKDF suppose que le secret d'entrée est déjà imprévisible. Il est donc conçu pour être **rapide** : la dernière ligne de la démo montre qu'on teste des milliers de candidats en quelques millisecondes. Un mot de passe humain a peu d'entropie : un attaquant qui a volé la base

essaierait des milliards de mots de passe courants. Pour les mots de passe, il faut une fonction **lente et salée** : PBKDF2 (§3.1), ou Argon2id, bcrypt, scrypt via une bibliothèque.

Entrée	Outil
Mot de passe choisi par un humain	PBKDF2, Argon2id, bcrypt, scrypt (**lents**)
Secret aléatoire ou issu d'un échange de clés	HKDF via l'API `KDF` (**rapide**)

3.3 🔒 Désérialisation Java : ne désérialisez jamais une donnée non fiable

Ce cours n'enseigne pas la sérialisation Java (`Serializable`, `ObjectOutputStream`, `ObjectInputStream`). Ce n'est pas un oubli : c'est un mécanisme **dangereux**, que ses concepteurs eux-mêmes considèrent comme une erreur historique. Vous le rencontrerez pourtant dans du code ancien, et vous devez savoir pourquoi il est dangereux.

🔒 **Sécurité — Désérialisation non fiable (CWE-502, OWASP A08).**

Le mécanisme. `ObjectInputStream.readObject()` reconstruit des objets à partir d'octets. Le flux d'octets **choisit lui-même** quelles classes instancier, parmi toutes celles du classpath. Pendant la reconstruction, la JVM appelle du code de ces classes (`readObject`, `hashCode`, `equals`, `compareTo`…).

L'attaque. Un attaquant fabrique un flux qui assemble des objets de bibliothèques **parfaitement légitimes** présentes dans votre classpath (Apache Commons Collections, Spring, Groovy…) : c'est une **chaîne de gadgets**. Chaque maillon appelle le suivant pendant la désérialisation, et le dernier exécute une commande système. Résultat : **exécution de code à distance (RCE)** avant même que votre code ne regarde l'objet reçu. L'outil `ysoserial` produit ces chaînes en une ligne de commande. Plusieurs failles majeures (serveurs d'applications Java, Jenkins, etc.) ont suivi ce schéma.

Autre risque. Même sans gadget, un flux de quelques kilo-octets peut décrire des structures imbriquées (`HashSet` dans `HashSet`…) dont la reconstruction prend des heures : c'est un **déni de service**.

Les protections, par ordre de préférence :

1. **Ne pas utiliser la sérialisation Java** pour des données venues de l'extérieur (réseau, fichier, cookie, cache partagé). Préférez un format de **données** : JSON (avec Jackson, vers des `record`), Protocol Buffers… Ces formats ne choisissent pas les classes à instancier.
2. Si vous ne pouvez pas l'éviter (code ancien) : un **`ObjectInputFilter`** en liste blanche (JEP 290, Java 9 ; filtres par contexte avec JEP 415, Java 17). Il refuse toute classe qui n'est pas explicitement autorisée, et limite la profondeur et la taille du graphe d'objets.
3. Garder les dépendances à jour, et retirer celles qui ne servent pas : moins de classes dans le classpath, moins de gadgets.

Pour un code ancien, le filtre se configure ainsi :

```
// Liste blanche : uniquement nos classes de données, profondeur et taille limitées ; tout le reste est refusé.
ObjectInputFilter filter = ObjectInputFilter.Config.createFilter(

"com.foodfast.legacy.OrderSnapshot;java.lang.String;java.math.*;maxdepth=5;maxbytes=10000;!*");

try (var in = new ObjectInputStream(new ByteArrayInputStream(receivedBytes))) {
    in.setObjectInputFilter(filter);   // AVANT le premier readObject()
    Object obj = in.readObject();      // une classe non listée lève InvalidClassException
}
```

On peut aussi définir le filtre pour toute la JVM, sans toucher au code : `java -Djdk.serialFilter='…;!*' …`.

> 🔒 Le `!*` final est essentiel : sans lui, le filtre ne refuse que ce qui est explicitement interdit, et c'est une liste noire. Une liste noire de gadgets est toujours en retard d'une chaîne.

---

4. Ce qu'il faut retenir

Concurrence

- Des threads partagent la mémoire : toute donnée **mutable et partagée** est un risque.
- `ExecutorService` dans un **try-with-resources** ; `newVirtualThreadPerTaskExecutor()` pour les tâches qui attendent (réseau, base). `close()` attend la fin des tâches.
- `count++` n'est **pas** atomique. Une race condition se démontre avec beaucoup d'opérations, et un test qui passe ne prouve rien.
- Corrections : `AtomicInteger` (une variable), `synchronized` (plusieurs variables, **même verrou pour lire et écrire**), `ConcurrentHashMap` avec ses opérations atomiques (`merge`, `compute`, `putIfAbsent`, `remove`). Le mieux : des objets **immuables**.
- 🔒 **TOCTOU** : « vérifier puis agir » sur un état partagé est une faille. La vérification et l'action doivent former **une seule opération atomique** (`remove`, `putIfAbsent`, `compute`, `CREATE_NEW`, contrainte SQL), ou être sous un verrou que **tous** les accès prennent.

JDBC

- `Connection`, `PreparedStatement`, `ResultSet` : toujours en **try-with-resources**.
- Le SQL dans des **text blocks**, les valeurs dans des `?`, le mapping dans un **repository** qui renvoie des objets (`Optional` pour une recherche par identifiant).
- 🔒 **Injection SQL** : jamais de concaténation d'une valeur dans du SQL. `PreparedStatement` pour les valeurs ; **liste blanche** pour les noms de colonnes.
- 🔒 **Secrets** hors du code (variables d'environnement, gestionnaire de secrets) ; compte applicatif au **moindre privilège**.

Pour les curieux : transactions (`setAutoCommit(false)`, `commit`, `rollback`), PBKDF2 avec sel et `MessageDigest.isEqual`, HKDF pour dériver des clés (jamais pour des mots de passe), scoped values, deadlocks, et jamais de désérialisation Java sur des données non fiables.

---

5. Pour aller plus loin

- **Concurrence**
  - dev.java, *Virtual Threads* : https://dev.java/learn/new-features/virtual-threads/
  - JEP 444, *Virtual Threads* : https://openjdk.org/jeps/444
  - JEP 491, *Synchronize Virtual Threads without Pinning* : https://openjdk.org/jeps/491
  - JEP 506, *Scoped Values* : https://openjdk.org/jeps/506
  - Javadoc `ConcurrentHashMap` (Java 25) : https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html
  - Livre de référence : Brian Goetz et al., *Java Concurrency in Practice*.
- **JDBC**
  - Tutoriel JDBC d'Oracle : https://docs.oracle.com/javase/tutorial/jdbc/
  - Javadoc `java.sql` (Java 25) : https://docs.oracle.com/en/java/javase/25/docs/api/java.sql/java/sql/package-summary.html
  - H2 : https://h2database.com
- **Sécurité**
  - OWASP, *SQL Injection Prevention Cheat Sheet* : https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
  - OWASP, *Password Storage Cheat Sheet* : https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
  - OWASP, *Deserialization Cheat Sheet* : https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html
  - CWE-367 (TOCTOU), CWE-89 (injection SQL), CWE-502 (désérialisation) : https://cwe.mitre.org
  - JEP 510, *Key Derivation Function API* : https://openjdk.org/jeps/510
  - SEI CERT Oracle Coding Standard for Java, chapitres VNA (visibilité et atomicité) et LCK (verrous) : https://wiki.sei.cmu.edu/confluence/display/java